1.2.3 SOS DP

上面枚举子集的方法有明显的缺陷：当一个状态的二进制位上有 k 个 0 时，它将在其他（不包含本身）状态迭代时被访问 2^k-1 次，存在重复的计算。

而产生这种现象的原因就是：我们没有在 $A[x]$ 被不同 $F[sta]$ 利用时建立一定的联系。我们应该加入另一个状态来避免上述的重复计算。

定义状态 $S(sta)=\{x|x\subseteq sta\}$。现在我们把这个集合划分为不相交的组。

$S(sta,i)=\{x|x\subseteq sta\&\&sta\oplus x<2^{i+1}\}$

我们将二进制位数从 0 开始从低位向高位表示，那集合 $S(sta,i)$ 就表示只有第 i 位以及更低位与 sta 不同的 x 的集合。

举个例子：$S(1011010,3)=\{1011010,1010010,1011000,1010000\}$。其中 sta 中的 1010 即为 sta 的第 3 至第 0 位，集合中的元素的加粗部分都与 sta 保持一致。

让我们尝试将 sta 与 x 建立联系。

1. sta 的第 i 位是 0。

显而易见地，sta 与 x 的第 i 位均为 0。因此 x 仅有 0 至 $i-1$ 位与 sta 不同，故有 $S(sta,i)=S(sta,i-1)$。

2. sta 的第 i 位是 1。

那么 $S(sta,i)$ 就有两部分组成：

- 第一部分：x 的第 i 位为 0，即为 $S(sta\oplus 2^i,i-1)$
- 第二部分：x 的第 i 位为 1，即为 $S(sta,i-1)$

请注意，这些关系形成一个有向无环图，而不一定是有根树（请考虑当 sta 不同但 i 相同时）

在实现了这些关系之后，我们可以很容易地写出相应的动态规划。

```
for(int sta=0;sta<(1<<N);sta++)
{
    dp[sta][-1]=A[sta];// 叶结点
    for(int i=0;i<N;i++)
    {
        if(sta&(1<<i))
            dp[sta][i]=dp[sta][i-1]+dp[sta^(1<<i)][i-1];
        else dp[sta][i]=dp[sta][i-1];
    }
    F[sta]=dp[sta][N-1];
}
```

空间优化

上图的算法浪费了较多空间，注意到每次 i 这一维都由 $i-1$ 转移而来，因此可以采取滚动数组的方式优化空间。

```
for(int i=0;i<(1<<N);i++)
    F[i]=A[i];
for(int i=0;i<N;i++)
    for(int sta=(1<<N)-1;sta>=0;sta--)
        if(sta&(1<<i))
            F[sta]+=F[sta^(1<<i)];
```

枚举顺序

值得注意的是，在此处 sta 这一维可以采用正序枚举的方式。

原因：.....

上述算法的时间复杂度即为 $O(N\cdot 2^N)$。

1.3 应用范围

其实 SOS DP 不仅可以求子集和，还可以求超集和。

超集和的核心代码如下：

```
for(int i = 0; i < N; i++)
  for(int sta = (1 << N) - 1; sta >= 0; sta--)
    if(!(sta & (1 << i)))
      F[sta] += F[sta ^ (1 << i)];
```

细心的读者会注意到，求子集和与求超集和之间的区别仅差一个！。

如何理解呢？这相当于我们把二进制中的每一个 1 当作 0 看待，0 当作 1 看待。比如 $sta=(1001)_2$，$i=2$，那 $F[(1001)_2]$ 就会从 $F[(1101)_2]$ 转移而来。而 sta 为 $(1101)_2$ 的子集，$(1101)_2$ 为 sta 的超集。